

PRACTICAL WEEK-11

Task-1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
GNU nano 8.7.1                                race.c *
#include <stdio.h>
#include <pthread.h>
|
#define THREADS 4
#define COUNT 100000

int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < COUNT; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Final counter = %d\n", counter);
    printf("Expected counter = %d\n", THREADS * COUNT);

    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify

```
root@Mani:~/practical$ cd practical
root@Mani:~/practical$ nano race.c
root@Mani:~/practical$ gcc race.c -o race -pthread
root@Mani:~/practical$ ./race
Final counter = 227368
Expected counter = 400000
root@Mani:~/practical$ ./race
Final counter = 400000
Expected counter = 400000
root@Mani:~/practical$ ./race
Final counter = 297551
Expected counter = 400000
root@Mani:~/practical$ ./race
Final counter = 228607
Expected counter = 400000
root@Mani:~/practical$
```

Report: A multithreaded counter application was implemented using POSIX threads. Multiple threads concurrently modified a shared counter without synchronization. Different executions produced inconsistent results due to race conditions. This demonstrated the need for synchronization when accessing shared data.

Task-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

```
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define COUNT 100000

int counter = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    for (int i = 0; i < COUNT; i++) {

        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main() {
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Final counter = %d\n", counter);
    printf("Expected counter = %d\n", THREADS * COUNT);

    pthread_mutex_destroy(&lock);
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^\ **Replace**

^K Cut
^U Paste

^T E
^J J

```
root@Mani:~/practical$ gcc mutex.c -o mutex -pthread
root@Mani:~/practical$ ./mutex
Final counter = 400000
Expected counter = 400000
root@Mani:~/practical$ ./mutex
Final counter = 400000
Expected counter = 400000
root@Mani:~/practical$ ./mutex
Final counter = 400000
Expected counter = 400000
root@Mani:~/practical$ ./mutex
Final counter = 400000
Expected counter = 400000
root@Mani:~/practical$
```

Report: The counter program was modified to use a POSIX mutex for synchronization. `pthread_mutex_lock()` and `pthread_mutex_unlock()` protected the shared counter from simultaneous access. Unlike the race-condition version, the synchronized program consistently produced the expected result. Synchronization improves correctness and reliability, although mutex operations introduce some execution overhead.